address_normalization_matching_poc_prompt.txt
TXT

ignore any previous session memory and consider below prompt file

Building a proof-of-concept for location parsing with libpostal.

I'll build the full POC as a runnable project, try to install libpostal here so the tests run against the real library, and send you the project as a zip.

Created 36 files, edited 3 files, and 44 more steps

Check Python, build tools, and installed packages
Setting up address parsing tools and dependencies.
Install Python dependencies
Now building native libpostal in the background (it's slow, and its data download may be blocked here).
Clone and configure libpostal from source
Setting up libpostal and checking its data download process.
Compile, download data for, and install libpostal
Wiring export generation, LLM queries, and configurable settings.
Added task **Build address-matching POC**
Added task **Install libpostal and run tests**
Added task **Write README and deliver**
Started task **Build address-matching POC**
Created **scoring.toml**
Created **__init__.py**
Created **settings.py**
Created **scoring_config.py**
Structuring code into separate normalization and component extraction modules.
Created **components.py**
Created **normalization.py**
Untangling how suffix tokens affect address normalization.
Edited **components.py**
Edited **normalization.py**
Designing field-by-field matching rules for addresses.
Created **comparison.py**
Defining decision rules and conflict-handling logic in detail.
Created **scoring.py**
Created **decision.py**
Created **explanation.py**
Comparing address fields to judge how well they match.
Created **matcher.py**
Created **excel_io.py**
Handling blank-row edge cases for column parsing.
Created **job_store.py**
Rewriting the query logic with cleaner SQL structure.
Edited **job_store.py**
Edited **job_store.py**
Created **workflow.py**
Ran a command
Created **jobs.py**
Check libpostal build, install, and data download result
The native library and its 1.9 GB of data installed. Now the Python binding.
Install pypostal binding and smoke-test it **Failed**
Inspect pypostal build error
Find the actual compile error
Resolving a setuptools packaging conflict on Debian.
Build pypostal in an isolated PEP 517 environment
Picking the right address expansion for display output.
Explore libpostal expansion options per component
Weighing heuristics to pick the best street-name expansion.
Add component-aware expansion and display preference
Building an address-matching tool using street-name patterns.
Try the matcher on sample pairs with real libpostal
Refining coverage rules and display formatting for edge cases.
Ran a command
Comparing postcode formatting across address components.
Ran a command
Debugging an unexpected empty search result

Debugging an unexpected empty search result.

Read explanation.py Failed ›

The patch misfired (empty-string replace); rewriting that file cleanly.

Created explanation.py Failed ›

Remove the corrupted explanation module ›

Created explanation.py

Rerun sample pairs ›

Matching behaves sensibly on real libpostal. Now the follow-up chat module and optional LLM client.

Created llm.py

Created chat.py

Detecting building number fields to flag mismatched rows.

Created streamlit_app.py

Debugging the uploader's rerun and button behavior.

Created create_sample_data.py

Created evaluate.py

Created install_libpostal.sh

Reviewing build configuration across platform-specific scripts and project files.

Get exact package versions for requirements ›

Created requirements.txt

Created requirements-dev.txt

Created pyproject.toml

Created .env.example

Created .gitignore

Created Dockerfile

Setting up test fixtures for the project.

Created conftest.py

Created test_matching.py

Testing scoring logic and address normalization for accuracy.

Created test_normalization.py

Created test_excel_io.py

Created test_workflow.py

Created test_chat.py

Created test_libpostal_integration.py

Fixing a flawed retry test's exception handling.

Run the full test suite ›

Inspect the failing test and the openpyxl warning ›

Debugging spreadsheet cell behavior for empty string values.

Ran a command ›

Verifying test coverage and preparing end-to-end sample runs.

Create sample workbook and run it through the real pipeline ›

Checking edge cases in data handling and normalization.

Run evaluation with threshold sweep ›

Testing the interface with an automated smoke test.

Drive the Streamlit app with AppTest ›

Fix deprecated arg and rerun the UI smoke test ›

Verifying the app serves correctly with a headless run.

Created test_ui_smoke.py

Debugging shared caching across test runs in the app.

Run all tests and start the app headlessly ›

Verifying test suite behavior without the optional dependency.

Verify suite behaviour when libpostal is absent ›

Identify the failure without libpostal ›

Show failure details ›

Check whether the failure is intermittent ›

One intermittent failure appeared; hunting it down.

Repeat job-related tests to reproduce flakiness ›

Investigating whether duplicate submissions produce identical output files.

Fix duplicate-submission test and repeat it ›

Completed task Build address-matching POC

Completed task Install libpostal and run tests

Started task Write README and deliver

Drafting a README with installation steps for multiple platforms.

Get libpostal version and list project files ›

Created README.md

Auditing file permission and logging claims for accuracy.

Final test run and package the project as a zip ›

Shared address_matcher_poc.zip

Completed task Write README and deliver

I've built the POC as described in your prompt file and sent it as `address_matcher_poc.zip`. I

I've built the POC as described in your prompt file and sent it as `address_matcher_poc.zip`. I tested it against the real libpostal, which I built from source here along with its 1.9 GB of data. All 85 tests passed. If libpostal is missing, its 15 tests skip and the other 70 still pass.

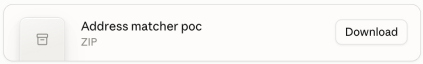
How it works: one LangGraph workflow runs the job: validate, process rows in batches, write the output file, record the counts. Every step is plain, tested Python; no LLM is involved in normalizing, scoring or deciding. Each job is stored in its own small SQLite database, so the UI can show progress and a failed job resumes from its last finished batch. The follow-up chat answers only from those stored results. An LLM is optional and is used only to understand oddly worded questions; it sees the question text only, never the addresses.

Results on the sample file (real output, saved as `data/sample/expected_output.xlsx`):

- `12 Main St` vs `21 Main St` → `NON-MATCH | Score: 67/100 | Building number differs: 12 vs 21; …`. A conflicting building number, unit, street or PO box decides the result whatever the score.
- A postal code that differs → REVIEW, because it could be a typo. You can change that in `config/scoring.toml`.
- A building number missing on one side, or no city or postal code to confirm location → REVIEW. Missing parts are never counted as mismatches.
- `Hauptstr.` / `Hauptstraße`, `5th` / `Fifth Ave`, `NW1 6XE` / `NW16XE` and `Apt #4-B` / `Apartment 4B` all came out as MATCH.

Things you should know:

- **Installing the Python binding:** on Debian/Ubuntu, `pip install postal` fails. Use `pip install --use-pep517 postal`. `scripts/install_libpostal.sh` and the README already include this.
- **City aliases:** `Bangalore` vs `Bengaluru` comes out as REVIEW because there's no alias table yet. That's the first thing I'd add for Indian data.
- **Tuning:** all weights and thresholds are starting guesses, and the score is not a probability. To tune them, label a few hundred of your own address pairs and run `scripts/evaluate.py --sweep`. My 15 synthetic labelled rows only show how the script works.
- **Not tested:** the macOS steps and the Dockerfile, since neither was available here. The README says so.
- **POC limits:** it runs as a single process with no login, and anyone who can open the app can see recent jobs. The README covers these and a path to production (Celery/Redis, PostgreSQL, FastAPI, sign-in).



Reply
Claude is AI and can make mistakes.
Opus 5.5 Medium
Manual